

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Model
model = Sequential([
    Dense(10, activation='softmax', input_shape=(784,))
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
model.summary()

model.fit(X_train, y_train, epochs=10, batch_size=64)

test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 0s 0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to the `Dense` layer constructor. Please use `DenseLayerConfig` instead.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 10)	7,850

Total params: 7,850 (30.66 KB)
Trainable params: 7,850 (30.66 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/10
938/938 2s 2ms/step - accuracy: 0.8568 - loss: 0.5532
Epoch 2/10
938/938 2s 2ms/step - accuracy: 0.9105 - loss: 0.3250
Epoch 3/10
938/938 3s 2ms/step - accuracy: 0.9173 - loss: 0.2965
Epoch 4/10
938/938 2s 2ms/step - accuracy: 0.9209 - loss: 0.2828
Epoch 5/10
938/938 2s 2ms/step - accuracy: 0.9236 - loss: 0.2743
Epoch 6/10
938/938 2s 2ms/step - accuracy: 0.9248 - loss: 0.2682
Epoch 7/10
938/938 2s 2ms/step - accuracy: 0.9268 - loss: 0.2641
Epoch 8/10
938/938 2s 2ms/step - accuracy: 0.9271 - loss: 0.2606
Epoch 9/10
938/938 3s 3ms/step - accuracy: 0.9291 - loss: 0.2575
Epoch 10/10
938/938 2s 2ms/step - accuracy: 0.9295 - loss: 0.2549
313/313 1s 2ms/step - accuracy: 0.9266 - loss: 0.2662
Test accuracy: 0.9265999794006348

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Model
model = Sequential([
```

```
Dense(10, activation='softmax', input_shape=(784,))
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

model.fit(X_train, y_train, epochs=20, batch_size=32)

test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_5 (Dense)	(None, 10)	7,850

Total params: 7,850 (30.66 KB)
Trainable params: 7,850 (30.66 KB)
Non-trainable params: 0 (0.00 B)

Epoch 1/20
1875/1875 ————— 4s 2ms/step - accuracy: 0.8773 - loss: 0.4650

Epoch 2/20
1875/1875 ————— 6s 2ms/step - accuracy: 0.9149 - loss: 0.3036

Epoch 3/20
1875/1875 ————— 4s 2ms/step - accuracy: 0.9209 - loss: 0.2836

Epoch 4/20
1875/1875 ————— 3s 2ms/step - accuracy: 0.9238 - loss: 0.2729

Epoch 5/20
1875/1875 ————— 6s 2ms/step - accuracy: 0.9252 - loss: 0.2671

Epoch 6/20
1875/1875 ————— 3s 2ms/step - accuracy: 0.9272 - loss: 0.2618

Epoch 7/20
1875/1875 ————— 3s 2ms/step - accuracy: 0.9292 - loss: 0.2579

Epoch 8/20
1875/1875 ————— 4s 2ms/step - accuracy: 0.9285 - loss: 0.2555

Epoch 9/20
1875/1875 ————— 4s 2ms/step - accuracy: 0.9301 - loss: 0.2530

Epoch 10/20
1875/1875 ————— 3s 2ms/step - accuracy: 0.9307 - loss: 0.2512

Epoch 11/20
1875/1875 ————— 3s 2ms/step - accuracy: 0.9309 - loss: 0.2492

Epoch 12/20
1875/1875 ————— 5s 2ms/step - accuracy: 0.9312 - loss: 0.2480

Epoch 13/20
1875/1875 ————— 3s 2ms/step - accuracy: 0.9315 - loss: 0.2465

Epoch 14/20
1875/1875 ————— 3s 2ms/step - accuracy: 0.9320 - loss: 0.2452

Epoch 15/20
1875/1875 ————— 4s 2ms/step - accuracy: 0.9325 - loss: 0.2438

Epoch 16/20
1875/1875 ————— 4s 2ms/step - accuracy: 0.9325 - loss: 0.2432

Epoch 17/20
1875/1875 ————— 3s 2ms/step - accuracy: 0.9333 - loss: 0.2422

Epoch 18/20
1875/1875 ————— 3s 2ms/step - accuracy: 0.9333 - loss: 0.2409

Epoch 19/20
1875/1875 ————— 4s 2ms/step - accuracy: 0.9339 - loss: 0.2404

Epoch 20/20
1875/1875 ————— 4s 2ms/step - accuracy: 0.9339 - loss: 0.2400

313/313 ————— 1s 2ms/step - accuracy: 0.9258 - loss: 0.2690

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Model
model = Sequential([
    Dense(10, activation='softmax', input_shape=(784,))
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

model.fit(X_train, y_train, epochs=15, batch_size=128)
```

```
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
dense_6 (Dense)	(None, 10)	7,850

Total params: 7,850 (30.66 KB)
Trainable params: 7,850 (30.66 KB)
Non-trainable params: 0 (0.00 B)

Epoch 1/15
469/469 2s 2ms/step - accuracy: 0.8359 - loss: 0.6754

Epoch 2/15
469/469 1s 2ms/step - accuracy: 0.9021 - loss: 0.3627

Epoch 3/15
469/469 1s 2ms/step - accuracy: 0.9115 - loss: 0.3196

Epoch 4/15
469/469 2s 4ms/step - accuracy: 0.9169 - loss: 0.2996

Epoch 5/15
469/469 2s 4ms/step - accuracy: 0.9201 - loss: 0.2875

Epoch 6/15
469/469 1s 2ms/step - accuracy: 0.9225 - loss: 0.2798

Epoch 7/15
469/469 1s 2ms/step - accuracy: 0.9241 - loss: 0.2735

Epoch 8/15
469/469 1s 3ms/step - accuracy: 0.9251 - loss: 0.2690

Epoch 9/15
469/469 1s 3ms/step - accuracy: 0.9261 - loss: 0.2652

Epoch 10/15
469/469 1s 3ms/step - accuracy: 0.9268 - loss: 0.2622

Epoch 11/15
469/469 1s 3ms/step - accuracy: 0.9278 - loss: 0.2596

Epoch 12/15
469/469 1s 3ms/step - accuracy: 0.9285 - loss: 0.2573

Epoch 13/15
469/469 2s 4ms/step - accuracy: 0.9290 - loss: 0.2552

Epoch 14/15
469/469 2s 4ms/step - accuracy: 0.9299 - loss: 0.2536

Epoch 15/15
469/469 2s 3ms/step - accuracy: 0.9301 - loss: 0.2521

313/313 1s 2ms/step - accuracy: 0.9269 - loss: 0.2618

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Input
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.datasets import mnist

# Load and preprocess MNIST
(X_train, y_train), (X_test, y_test) = mnist.load_data()
X_train, X_test = X_train / 255.0, X_test / 255.0
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# parameters requested:
# Neurons: 32, 64, 128 | Layers: +2 | Batches: 32, 64, 128 | Epochs: 10, 15, 20

model_deep = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu'), # Increased neurons
    Dense(64, activation='relu'), # Extra layer 1
    Dense(32, activation='relu'), # Extra layer 2
    Dense(10, activation='softmax')
])

model_deep.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Training with increased epochs and batch size
model_deep.fit(X_train, y_train, epochs=20, batch_size=128, validation_split=0.1)

# Evaluate
test_loss, test_acc = model_deep.evaluate(X_test, y_test)
print(f'Test Accuracy with deeper model: {test_acc:.4f}')
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input\_shape`/`input\_dim`
super().__init__(**kwargs)

Epoch 1/20
422/422 4s 6ms/step - accuracy: 0.8909 - loss: 0.3803 - val_accuracy: 0.9600 - val_loss: 0.1449

Epoch 2/20
422/422 2s 5ms/step - accuracy: 0.9596 - loss: 0.1378 - val_accuracy: 0.9672 - val_loss: 0.1087

Epoch 3/20
422/422 4s 8ms/step - accuracy: 0.9720 - loss: 0.0952 - val_accuracy: 0.9727 - val_loss: 0.0901

Epoch 4/20
422/422 2s 6ms/step - accuracy: 0.9787 - loss: 0.0716 - val_accuracy: 0.9760 - val_loss: 0.0838

Epoch 5/20
422/422 3s 6ms/step - accuracy: 0.9828 - loss: 0.0567 - val_accuracy: 0.9748 - val_loss: 0.0869

Epoch 6/20
422/422 2s 5ms/step - accuracy: 0.9857 - loss: 0.0471 - val_accuracy: 0.9733 - val_loss: 0.0893

Epoch 7/20
422/422 2s 5ms/step - accuracy: 0.9881 - loss: 0.0375 - val_accuracy: 0.9798 - val_loss: 0.0681

Epoch 8/20
422/422 3s 8ms/step - accuracy: 0.9905 - loss: 0.0305 - val_accuracy: 0.9813 - val_loss: 0.0791

Epoch 9/20
422/422 2s 5ms/step - accuracy: 0.9920 - loss: 0.0257 - val_accuracy: 0.9768 - val_loss: 0.0932

Epoch 10/20
422/422 2s 5ms/step - accuracy: 0.9923 - loss: 0.0237 - val_accuracy: 0.9775 - val_loss: 0.0881

Epoch 11/20
422/422 2s 5ms/step - accuracy: 0.9930 - loss: 0.0221 - val_accuracy: 0.9767 - val_loss: 0.0926
Epoch 12/20
422/422 2s 5ms/step - accuracy: 0.9939 - loss: 0.0183 - val_accuracy: 0.9778 - val_loss: 0.0952
Epoch 13/20
422/422 3s 8ms/step - accuracy: 0.9956 - loss: 0.0143 - val_accuracy: 0.9797 - val_loss: 0.0853
Epoch 14/20
422/422 2s 6ms/step - accuracy: 0.9962 - loss: 0.0122 - val_accuracy: 0.9777 - val_loss: 0.0973
Epoch 15/20
422/422 2s 5ms/step - accuracy: 0.9942 - loss: 0.0171 - val_accuracy: 0.9777 - val_loss: 0.0985
Epoch 16/20
422/422 2s 5ms/step - accuracy: 0.9966 - loss: 0.0103 - val_accuracy: 0.9775 - val_loss: 0.0994
Epoch 17/20
422/422 2s 5ms/step - accuracy: 0.9969 - loss: 0.0093 - val_accuracy: 0.9792 - val_loss: 0.0942
Epoch 18/20
422/422 3s 7ms/step - accuracy: 0.9957 - loss: 0.0128 - val_accuracy: 0.9792 - val_loss: 0.1019
Epoch 19/20
422/422 3s 7ms/step - accuracy: 0.9972 - loss: 0.0092 - val_accuracy: 0.9775 - val_loss: 0.1064
Epoch 20/20
422/422 2s 5ms/step - accuracy: 0.9956 - loss: 0.0123 - val_accuracy: 0.9772 - val_loss: 0.1163
313/313 1s 2ms/step - accuracy: 0.9776 - loss: 0.1105
Test Accuracy with deeper model: 0.9776

```
import numpy as np
from tensorflow.keras.datasets import mnist

# Load dataset to count
(X_train, y_train), (X_test, y_test) = mnist.load_data()

print(f'Total images in MNIST training set: {len(X_train)}')
print(f'Total images in MNIST test set: {len(X_test)}')
print(f'Grand total: {len(X_train) + len(X_test)}')
```

Total images in MNIST training set: 60000
Total images in MNIST test set: 10000
Grand total: 70000

```
from google.colab import files
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Upload your images from C:\Users\heman\Desktop\dataset 0-9
print('Please upload the images from your local folder:')
uploaded = files.upload()

# Step 2: Process and Predict
for filename in uploaded.keys():
    # Read image as grayscale
    img = cv2.imdecode(np.frombuffer(uploaded[filename], np.uint8), cv2.IMREAD_GRAYSCALE)

    # Check if image was loaded successfully
    if img is None:
        print(f"Error: Could not decode image from file {filename}. Please ensure it is a valid image file (e.g., .jpg, .png).")
        continue

    # Resize to 28x28 (MNIST size) and invert colors if necessary (MNIST is white on black)
    img_resized = cv2.resize(img, (28, 28))
    img_normalized = img_resized / 255.0
    # Reshape to (1, 28, 28, 1) for a model expecting 4D input (batch, height, width, channels)
    img_input = img_normalized.reshape(1, 28, 28, 1)

    # Prediction
    prediction = model.predict(img_input, verbose=0)
    predicted_digit = np.argmax(prediction)
    confidence = np.max(prediction) * 100

    # Display
    plt.imshow(img_resized, cmap='gray')
    plt.title(f'File: {filename}\nPred: {predicted_digit}, Confidence: {confidence:.2f}%')
    plt.axis('off')
    plt.show()
    print(f'Image: {filename} -> Predicted: {predicted_digit} ({confidence:.2f}%')
```

Please upload the images from your local folder:

Choose Files

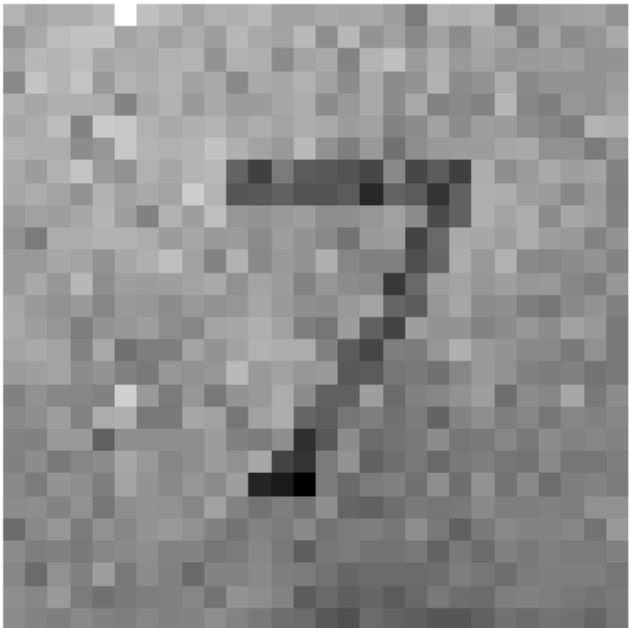
No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving WhatsApp Image 2026-04-27 at 3.02.12 AM.jpeg to WhatsApp Image 2026-04-27 at 3.02.12 AM.jpeg

File: WhatsApp Image 2026-04-27 at 3.02.12 AM.jpeg

Pred: 7, Confidence: 25.90%



Start coding or execute with AT